

CIS022 Day 12

standing at the very threshold of object orientation

Agenda

- Array review
- Enums
- Structs
- Your first class!

A note on where we are – this class moves *fast*

If you've done object-oriented programming before

- By the end of today, we'll be (mostly) finished with the C++ concepts that come *before* object-oriented programming.
- *You have a leg up on your fellow students!* Most of what you see from here on is new syntax.

If you haven't

- Most of the rest of the semester will be conceptually *very* new. We'll be learning a whole new programming paradigm.



The data structure we've seen

Arrays are a primitive data structure that can store many items of the *same type*.

Here, **list** holds five **strings**, and the loop visits each one by its index.

```
6  const int LIST_SIZE = 5;
7
8  int main() {
9      string list[LIST_SIZE] = {"ben", "evan", "keiran",
10                             "thana", "paul"};
11      for (int index = 0; index < LIST_SIZE; index++) {
12          cout << "current index: " << index << endl;
13          cout << "item at index: " << list[index] << endl;
14      }
15      return 0;
16 }
```

A couple of notes about arrays

Consider this short program that builds and prints a multiplication table. Some things to notice:

When you **declare** an array, give its size in brackets.

For a multidimensional array, give each additional size in its own set of brackets.

```
5  const int ARRAY_SIZE = 5;
6
7  void arrayInitializer(int array[][ARRAY_SIZE]) {
8      for (int row = 0; row < ARRAY_SIZE; row++) {
9          for (int column = 0; column < ARRAY_SIZE; column++) {
10             array[row][column] = row * column;
11         }
12     }
13 }
14
15 int main() {
16     int twoD[ARRAY_SIZE][ARRAY_SIZE];
17     arrayInitializer(twoD);
```

A couple of notes about arrays

Consider this short program that builds and prints a multiplication table. Some things to notice:

When you **pass** an array (of any dimension) into a function, just give its name.

No **&** needed: the function works on the original array, not a copy. That's how `arrayInitializer()` fills in `twoD`.

```
5  const int ARRAY_SIZE = 5;
6
7  void arrayInitializer(int array[][ARRAY_SIZE]) {
8      for (int row = 0; row < ARRAY_SIZE; row++) {
9          for (int column = 0; column < ARRAY_SIZE; column++) {
10             array[row][column] = row * column;
11         }
12     }
13 }
14
15 int main() {
16     int twoD[ARRAY_SIZE][ARRAY_SIZE];
17     arrayInitializer(twoD);
```

A couple of notes about arrays

Consider this short program that builds and prints a multiplication table. Some things to notice:

When an array is a **parameter**, show that it's an array with brackets.

For a one-dimensional array the brackets are empty:

```
int array[]
```

For a two-dimensional array, you must give the number of *columns* in the second set of brackets:

```
int array[][ARRAY_SIZE]
```

```
5  const int ARRAY_SIZE = 5;
6
7  void arrayInitializer(int array[][ARRAY_SIZE]) {
8      for (int row = 0; row < ARRAY_SIZE; row++) {
9          for (int column = 0; column < ARRAY_SIZE; column++) {
10             array[row][column] = row * column;
11         }
12     }
13 }
14
15 int main() {
16     int twoD[ARRAY_SIZE][ARRAY_SIZE];
17     arrayInitializer(twoD);
```

Running the multiplication table

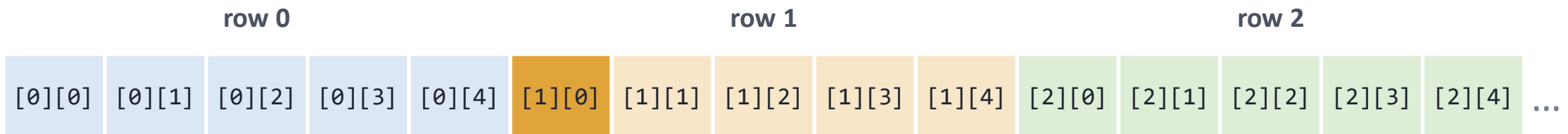
```
15  int main() {
16      int twoD[ARRAY_SIZE][ARRAY_SIZE];
17      arrayInitializer(twoD);
18
19      for (int row = 0; row < ARRAY_SIZE; row++) {
20          for (int column = 0; column < ARRAY_SIZE; column++) {
21              cout << twoD[row][column] << "\t";
22          }
23          cout << endl;
24      }
25
26      return 0;
27  }
```

The nested loop prints one row of the table per line, with a tab (`\t`) between columns.

```
$ ./multiply
0  0  0  0  0
0  1  2  3  4
0  2  4  6  8
0  3  6  9 12
0  4  8 12 16
```

A debugging tip: 2D arrays in memory

All the rows of a two-dimensional array are stored *one right after another* in memory. If you write to a column off the right edge of one row, you may end up writing to the left edge of the **next** row.



`twoD[0][5]` doesn't exist... but this is the memory it lands on: `twoD[1][0]`

Officially, reading or writing outside an array is *undefined behavior*: the program might quietly overwrite a neighbor (like above), crash, or seem fine until it suddenly isn't.

The way to avoid this bug? Make *super sure* you never read or write off the end of an array.

Structs and enums

we're almost to classes, everyone

Your first struct

Say you're working on a project with a bunch of *different* types of data, but where it makes sense to bundle those pieces of data together.

For example, say you want to store this information about a robot:

You can't store these together in one array. Row and column are **ints**, and the direction could be an **int**... but the name can't really be reduced to an **int**.

the row it's on **int**

the column it's on **int**

the direction it's facing **int?**

its name **string**

Your first struct

You *could* store them in separate variables...
but then you'd have to pass every one of
them into every function that uses them.

This is a pain.

Fortunately, C++ has a tool that lets us
bundle them all together: the **struct**.

```
int karelRow = 14;  
int karelColumn = 0;  
int karelFacing = 1;  
string karelName = "Karel T. Robot";  
  
karelMove(karelRow, karelColumn, karelFacing);  
karelPrint(karelRow, karelColumn, karelFacing,  
           karelName);
```

Your first struct

Here's a struct that bundles all four together.

We'll talk about what this means in a second

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
```

This struct stores four variables: two **ints**,
a **Direction**, and a **string**

```
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Tangent: enums

You might be wondering: what on earth is this **enum** thing?

Think of an enum as a way to bundle many related constants together.

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Tangent: enums

Each item in an enum is associated with an **int**.

By default the first one is associated with 0, the second with 1, and so forth: **NORTH** is 0, **EAST** is 1, **SOUTH** is 2, **WEST** is 3.

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Tangent: enums

This is the **name** of the enum.

Once you've defined it, **Direction** is a data type, just like **int** and **string** are data types.

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Tangent: enums

Once you've defined it, **Direction** is a data type, just like **int** and **string** are data types.

So here, inside the struct, I can declare a variable of type **Direction**.

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Tangent: enums

When I print a variable of type **Direction**, I get the integer that each item is associated with.

```
$ ./enums
```

```
first: 0
```

```
second: 1
```

```
third: 2
```

```
fourth: 3
```

```
10  int main() {  
11      Direction firstEnum = NORTH;  
12      Direction secondEnum = EAST;  
13      Direction thirdEnum = SOUTH;  
14      Direction fourthEnum = WEST;  
15  
16      cout << "first: " << firstEnum << endl;  
17      cout << "second: " << secondEnum << endl;  
18      cout << "third: " << thirdEnum << endl;  
19      cout << "fourth: " << fourthEnum << endl;  
20  
21      return 0;  
22  }
```

Tangent: enums

These two are *roughly* equivalent: both give you named constants for the four directions.

Option A: constants

```
const int NORTH = 0;  
const int EAST  = 1;  
const int SOUTH = 2;  
const int WEST  = 3;
```

Option B: an enum

```
enum Direction {  
    NORTH, EAST, SOUTH, WEST  
};
```

The difference: only the enum gives you a new *type*, so you can later create variables of type **Direction**.

Pick one. They can't both be in the same program: **NORTH**, **EAST**, **SOUTH**, and **WEST** would each be declared twice, and the compiler will complain.

Tangent: naming your types

By convention, when you define your own type, start its name with a capital letter: **Direction**, **Karel**.

This is **SUPER IMPORTANT** and actually required by the style guide. But not *just* because it's standard practice: your code is easier to read and debug when a capitalized name like **Karel** always means a type.

(Constants like **NORTH** and **ARRAY_SIZE** are written in ALL CAPS, so they're easy to tell apart.)

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Printing an enum class

Values of an `enum class` don't silently turn into `ints`, so `cout << facing` won't compile.

Instead, write a function that turns each value into a `string` using `if/else`. (A `switch` works just as well.)

Once we get to **operator overloading**, we'll be able to teach `cout` how to print a `Direction` directly, so `cout << facing` will just work.

```
enum class Direction { NORTH, EAST, SOUTH, WEST };
```

```
string directionName(Direction d) {  
    if (d == Direction::NORTH) {  
        return "north";  
    } else if (d == Direction::EAST) {  
        return "east";  
    } else if (d == Direction::SOUTH) {  
        return "south";  
    } else {  
        return "west";  
    }  
}
```

```
Direction facing = Direction::EAST;  
cout << directionName(facing) << endl;
```

```
$ ./directions  
east
```

Printing an enum class

Values of an `enum class` don't silently turn into `ints`, so `cout << facing` won't compile.

Instead, write a function that turns each value into a `string` using a `switch`. Each `case` returns right away, so no `break` is needed.

The `return` after the `switch` keeps the compiler from warning that the function might end without returning anything.

Once we get to **operator overloading**, we'll be able to teach `cout` how to print a `Direction` directly, so `cout << facing` will just work.

```
$ ./directions
east
```

```
enum class Direction { NORTH, EAST, SOUTH, WEST };
```

```
string directionName(Direction d) {
    switch (d) {
        case Direction::NORTH:
            return "north";
        case Direction::EAST:
            return "east";
        case Direction::SOUTH:
            return "south";
        case Direction::WEST:
            return "west";
    }
    return "unknown";
}
```

```
Direction facing = Direction::EAST;
cout << directionName(facing) << endl;
```

Back to structs

Let's return to structs.

Structs are another (much more flexible) way to define your own types.

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Back to structs

This struct contains four pieces of data, which happen to be of three different types: `int`, `Direction`, and `string`.

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Back to structs

Remember to put a **;** after the closing brace of a struct definition.

(The same goes for enum definitions.)

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  enum Direction {
7      NORTH, EAST, SOUTH, WEST
8  };
9
10 struct Karel {
11     int row;
12     int column;
13     Direction facing;
14     string name;
15 };
```

Creating a struct variable

You create a variable of a struct type exactly the way you create a variable of any other type: the name of the type, followed by the name you want to give this particular variable.

Here I create a variable of type `Karel` and name it `myFirstKarel`.

```
29  int main() {  
30      Karel myFirstKarel;
```

Assigning values inside a struct

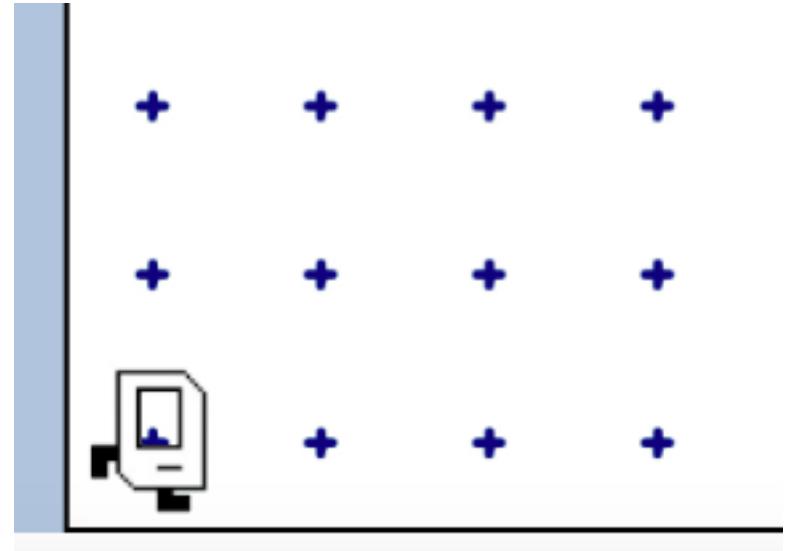
To assign a value to a variable inside a struct, give the name of the struct variable, then a dot, then the name of the variable inside the struct you want to change.

```
29  int main() {  
30      Karel myFirstKarel;  
31  
32      myFirstKarel.row = 14;  
33      myFirstKarel.column = 0;  
34      myFirstKarel.facing = EAST;  
35      myFirstKarel.name = "Karel T. Robot";
```

Assigning values inside a struct

Now we have a fully initialized variable of type `Karel`.

```
29  int main() {  
30      Karel myFirstKarel;  
31  
32      myFirstKarel.row = 14;  
33      myFirstKarel.column = 0;  
34      myFirstKarel.facing = EAST;  
35      myFirstKarel.name = "Karel T. Robot";  
}
```



Facing east, standing at row 14, column 0.

Rows are numbered from the top, so row 14 is the bottom row of Karel's world.

Passing structs to functions

We can now pass our `Karel` into functions.
No more passing a row, a column, a direction, and a name separately!

```
17 void karelMove(Karel &karel) {  
18     if (karel.facing == NORTH) {  
19         karel.row -= 1;  
20     } else if (karel.facing == WEST) {  
21         karel.column -= 1;  
22     } else if (karel.facing == SOUTH) {  
23         karel.row += 1;  
24     } else if (karel.facing == EAST) {  
25         karel.column += 1;  
26     }  
27 }
```

Passing structs to functions

Notice that we pass our `Karel` in **by reference** (`Karel &karel`).

That's because this function *changes* the `Karel` variable, and we want those changes to stick after the function returns.

```
17 void karelMove(Karel &karel) {  
18     if (karel.facing == NORTH) {  
19         karel.row -= 1;  
20     } else if (karel.facing == WEST) {  
21         karel.column -= 1;  
22     } else if (karel.facing == SOUTH) {  
23         karel.row += 1;  
24     } else if (karel.facing == EAST) {  
25         karel.column += 1;  
26     }  
27 }
```

How should you pass a struct?

Structs can get large, so copying them on every function call is wasteful. A good rule of thumb:

The function changes the struct
pass by reference

```
void karelMove(Karel &karel);
```

The function only reads the struct
pass by const reference

```
void karelPrint(const Karel &karel);
```

const means no copy *and* the compiler stops you if you accidentally try to change it.

Arrays are different: they're never copied when you pass them, so they never need the **&**.

Passing structs to functions

After we call `karelMove()`, `myFirstKarel.column` really has changed:

```
29  int main() {
30      Karel myFirstKarel;
31
32      myFirstKarel.row = 14;
33      myFirstKarel.column = 0;
34      myFirstKarel.facing = EAST;
35      myFirstKarel.name = "Karel T. Robot";
36
37      karelMove(myFirstKarel);
38      cout << "Karel's row: " << myFirstKarel.row << endl;
39      cout << "Karel's column: " << myFirstKarel.column << endl;
40
41      return 0;
42  }
```

```
$ ./karel
Karel's row: 14
Karel's column: 1
```

`column` is now 1 instead of 0.

Beyond structs

Notice that we have a function that only makes sense when it's applied to a `Karel`.

`karelMove()` is meaningless unless you've defined a `Karel` struct.

```
17 void karelMove(Karel &karel) {  
18     if (karel.facing == NORTH) {  
19         karel.row -= 1;  
20     } else if (karel.facing == WEST) {  
21         karel.column -= 1;  
22     } else if (karel.facing == SOUTH) {  
23         karel.row += 1;  
24     } else if (karel.facing == EAST) {  
25         karel.column += 1;  
26     }  
27 }
```

Your first class

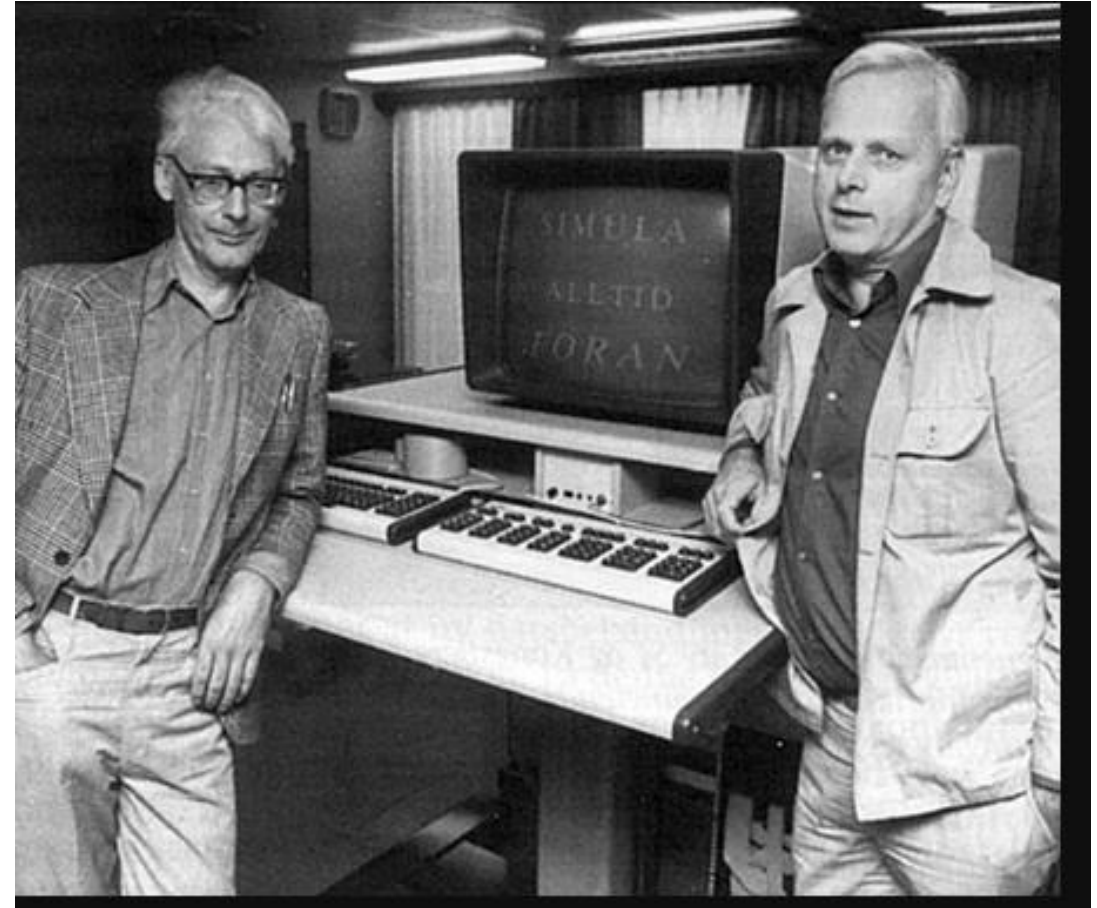
welcome to object-oriented programming

Welcome to object-oriented programming

In the 1960s, two Norwegian computer scientists, **Ole-Johan Dahl** and **Kristen Nygaard**, had an idea: bundle a data structure (like a **struct**) together with the functions that operate on that data.

Their language, Simula, was the first object-oriented language.

Bjarne Stroustrup borrowed Simula's classes when he created C++ (originally called “C with Classes”).



Ole-Johan Dahl (left) and Kristen Nygaard (right)

Welcome to object-oriented programming

C++ has a tool much like the struct that can hold not just variables, but also *functions* that operate on those variables.

It's called the **class**, and it's what we'll be talking about for literally the rest of the semester.

struct

Members are public by default

By convention: plain bundles of data

class

Members are private by default

By convention: types that also have functions

Technically, C++ structs can hold functions too. The default access is the only real language difference.

Your first class

Here's our `Karel` type defined using a `class` rather than a `struct`.

Notice it contains a function, `move()`, as well as variables.

```
10  class Karel {  
11  public:  
12      int row;  
13      int column;  
14      Direction facing;  
15      string name;  
16      void move();  
17  };
```

Your first class

Thing to notice #1: defining a class looks just like defining a struct, except you use the **class** keyword instead of **struct**.

```
10  class Karel {  
11  public:  
12      int row;  
13      int column;  
14      Direction facing;  
15      string name;  
16      void move();  
17  };
```

Your first class

Everything after **public:** can be accessed from *outside* the class, like from **main()**.

We need it because class members are **private** by default. Without it, **main()** couldn't touch **row**, **column**, and so on.

We'll talk (much, much) more about public vs. private, and why you'd ever want private, in future sessions.

```
10  class Karel {  
11  public:  
12      int row;  
13      int column;  
14      Direction facing;  
15      string name;  
16      void move();  
17  };
```

Your first class

And here's the version with `move()` actually defined.

We write the name of the class before the name of the function, separated by `::`

The return type goes before all of that, as usual.

```
10  class Karel {
11  public:
12      int row;
13      int column;
14      Direction facing;
15      string name;
16      void move();
17  };
18
19  void Karel::move() {
20      if (facing == NORTH) {
21          row -= 1;
22      } else if (facing == WEST) {
23          column -= 1;
24      } else if (facing == SOUTH) {
25          row += 1;
26      } else if (facing == EAST) {
27          column += 1;
28      }
29  }
```

Struct version vs. class version

struct version

```
17 void karelMove(Karel &karel) {
18     if (karel.facing == NORTH) {
19         karel.row -= 1;
20     } else if (karel.facing == WEST) {
21         karel.column -= 1;
22     } else if (karel.facing == SOUTH) {
23         karel.row += 1;
24     } else if (karel.facing == EAST) {
25         karel.column += 1;
26     }
27 }
```

class version

```
19 void Karel::move() {
20     if (facing == NORTH) {
21         row -= 1;
22     } else if (facing == WEST) {
23         column -= 1;
24     } else if (facing == SOUTH) {
25         row += 1;
26     } else if (facing == EAST) {
27         column += 1;
28     }
29 }
```

In the struct version, we have to pass in a `Karel` and say *which* Karel's variables to change: `karel.row`, `karel.column`.

Struct version vs. class version

struct version

```
17 void karelMove(Karel &karel) {
18     if (karel.facing == NORTH) {
19         karel.row -= 1;
20     } else if (karel.facing == WEST) {
21         karel.column -= 1;
22     } else if (karel.facing == SOUTH) {
23         karel.row += 1;
24     } else if (karel.facing == EAST) {
25         karel.column += 1;
26     }
27 }
```

class version

```
19 void Karel::move() {
20     if (facing == NORTH) {
21         row -= 1;
22     } else if (facing == WEST) {
23         column -= 1;
24     } else if (facing == SOUTH) {
25         row += 1;
26     } else if (facing == EAST) {
27         column += 1;
28     }
29 }
```

In the class version, we never say which Karel we're operating on. It's just **row** and **column**. So whose **row** is this?

Whose row is it?

A member function always runs on a particular object: the one **before the dot** when you call it.

```
myFirstKarel.move();
```



inside move(), this time:

row means myFirstKarel.row

column means myFirstKarel.column

facing means myFirstKarel.facing

So **myFirstKarel.move()** does the same job as **karelMove(myFirstKarel)** did. The object gets passed along for you.

(Behind the scenes, C++ hands the function a hidden pointer to that object, called **this**. More on that later.)

Using the Karel class

Here's a `main()` that uses the `Karel` class. Notice how we call `move()`: the same way we access a variable inside an object. Object's name, a dot, then the function.

```
31  int main() {
32      Karel myFirstKarel;
33
34      myFirstKarel.row = 14;
35      myFirstKarel.column = 0;
36      myFirstKarel.facing = EAST;
37      myFirstKarel.name = "Karel T. Robot";
38
39      myFirstKarel.move();
40      cout << "Karel's row: " << myFirstKarel.row << endl;
41      cout << "Karel's column: " << myFirstKarel.column << endl;
42
43      return 0;
44  }
```

Using the Karel class

Here's the output. Karel moved one column to the east:

```
31  int main() {  
32      Karel myFirstKarel;  
33  
34      myFirstKarel.row = 14;  
35      myFirstKarel.column = 0;  
36      myFirstKarel.facing = EAST;  
37      myFirstKarel.name = "Karel T. Robot";  
38  
39      myFirstKarel.move();  
40      cout << "Karel's row: " << myFirstKarel.row << endl;  
41      cout << "Karel's column: " << myFirstKarel.column << endl;  
42  
43      return 0;  
44  }
```

```
$ ./karel  
Karel's row: 14  
Karel's column: 1
```

Using the Karel class

Here's a version with two **Karel** variables: Karel and Lerak (Lerak is Karel's evil twin).

```
31  int main() {
32      Karel myFirstKarel;
33      myFirstKarel.row = 14;
34      myFirstKarel.column = 0;
35      myFirstKarel.facing = EAST;
36      myFirstKarel.name = "Karel T. Robot";
37
38      Karel mySecondKarel;
39      mySecondKarel.row = 14;
40      mySecondKarel.column = 0;
41      mySecondKarel.facing = EAST;
42      mySecondKarel.name = "Lerak (Karel's evil twin)";
```

Using the Karel class

Here's a version with two `Karel` variables: Karel and Lerak (Lerak is Karel's evil twin). I call `move()` on `myFirstKarel` but not on `mySecondKarel`.

```
44     myFirstKarel.move();
45     cout << "Karel's row: " << myFirstKarel.row << endl;
46     cout << "Karel's column: " << myFirstKarel.column << endl;
47
48     cout << "Lerak's row: " << mySecondKarel.row << endl;
49     cout << "Lerak's column: " << mySecondKarel.column << endl;
50
51     return 0;
52 }
```

Using the Karel class

I call `move()` on `myFirstKarel` but not on `mySecondKarel`... so Karel moves, but Lerak bides her time.

```
44     myFirstKarel.move();
45     cout << "Karel's row: " << myFirstKarel.row << endl;
46     cout << "Karel's column: " << myFirstKarel.column << endl;
47
48     cout << "Lerak's row: " << mySecondKarel.row << endl;
49     cout << "Lerak's column: " << mySecondKarel.column << endl;
50
51     return 0;
52 }
```

```
$ ./karel
Karel's row: 14
Karel's column: 1
Lerak's row: 14
Lerak's column: 0
```

Terminology

A variable whose type is a class is called an **object** (hence “object-oriented programming”).

This program has two objects, `myFirstKarel` and `mySecondKarel`. Both are **instances** of the `Karel` class.

```
class Karel
```

```
    row  
    column  
    facing  
    name  
    move()
```

```
myFirstKarel
```

```
    row = 14    column = 1  
    facing = EAST  
    name = "Karel T. Robot"
```

```
mySecondKarel
```

```
    row = 14    column = 0  
    facing = EAST  
    name = "Lerak (Karel's evil twin)"
```

the class: a description of a type

Next time

More on classes and object-oriented programming (and pretty much every session for the next seven weeks)